



Analyse & Améliorations

Soundboard VTApp

Analyse technique, problèmes identifiés et propositions d'amélioration

Document généré le 22 mai 2026

Table des matières

1. Architecture actuelle
2. Points forts
3. Problèmes identifiés
4. Correctifs urgents
5. Améliorations proposées
6. Feuille de route priorisée

1. Architecture actuelle

1.1 Stack technique

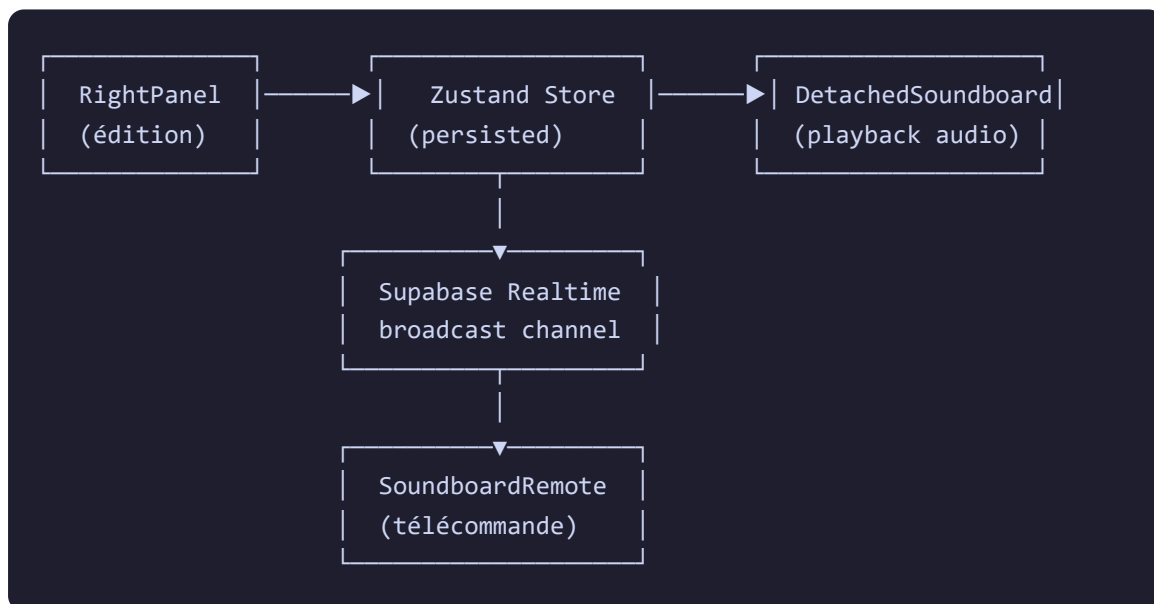
Couche	Technologie
Framework	React 19 + TypeScript 5.9
État	Zustand 5 + Zundo (undo/redo)
Realtime	Supabase Realtime (Broadcast)
Audio	HTMLAudioElement + Data URLs (base64)
Icônes	lucide-react
Build	Vite 7

1.2 Structure des fichiers

Fichier	Rôle	Lignes
<code>src/types/index.ts</code>	Interfaces <code>SoundButton</code> & <code>SoundboardState</code>	~35 (Soundboard)
<code>src/store/index.ts</code>	État Zustand : <code>setSoundboard</code> , <code>updateSoundButton</code> , <code>removeSoundButton</code>	~40
<code>src/components/DetachedSoundboard.tsx</code>	Fenêtre flottante de la soundboard (playback, drag)	281
<code>src/components/DetachedSoundboard.css</code>	Styles de la fenêtre flottante	96

<code>src/components/layout/RightPanel.tsx</code>	Soundboard intégrée au panneau latéral	~60 (section)
<code>src/components/EditingModal.tsx</code>	Modal d'édition des boutons sonores	~200
<code>src/pages/SoundboardRemote.tsx</code>	Télécommande mobile (5 onglets)	669
<code>src/pages/SoundboardJoin.tsx</code>	Page de connexion à la télécommande	87
<code>src/lib/realtime-host.ts</code>	Sync Realtime : écoute <code>soundboard_action</code> , broadcast état	~30
<code>src/lib/timer-sound.ts</code>	Utilitaire de beep audio	34

1.3 Flux de données



1.4 Modèle de données

```

SoundButton {
  index: number;      // Position dans la grille (0..N)
  name: string;       // Nom affiché
  audioUrl: string;   // Données audio en base64
}
  
```

```
isOneShot: boolean;    // false = boucle
icon?: string;         // Nom d'icône lucide-react
color?: string;        // Couleur du bouton
imageUrl?: string;     // Image de fond
volume?: number;       // Volume (0.0 - 1.0)
}

SoundboardState {
  cols, rows: number;  // Dimensions de la grille
  isDetached: boolean; // Fenêtre flottante ?
  x, y: number;        // Position de la fenêtre
  buttons: SoundButton[];
  remoteEnabled: boolean;
  remotePasscode: string;
  remoteShowSounds, remoteShowTasks, ... : boolean;
  remotePlayTrigger: { index, timestamp } | null;
}
```

2. Points forts

✓ Architecture Realtime bien pensée

La séparation hôte ↔ télécommande via Supabase Broadcast est élégante. Le passcode protège l'accès, et chaque onglet peut être masqué individuellement. L'optimistic update côté remote donne une sensation d'instantanéité.

✓ Interface fluide et moderne

Le drag & drop de la fenêtre via Pointer Events, les animations de glow pendant la lecture, et la progress bar en bas de chaque bouton sont des détails soignés. La télécommande mobile est complète (5 onglets) avec une navigation en bas très professionnelle.

✓ Gestion du cycle de vie audio

Le nettoyage des AudioElements au démontage du composant et la synchronisation des volumes via useEffect sont bien vus. Le support des boucles (looping) avec indicateur visuel est fonctionnel.

✓ Expérience mobile complète

La télécommande ne se limite pas aux sons : checklist, actions, handouts, joueurs — tout est accessible. Le design responsive avec dégradé et thème sombre est de qualité.

3. Problèmes identifiés

3.1 Tableau récapitulatif

#	Problème	Fichier	Criticité
1	Stockage base64 dans Zustand persisté — Les fichiers audio encodés en base64 sont stockés dans localStorage via le middleware persist. Un son de 3 Mo = ~4 Mo en base64. Avec 12 boutons, le localStorage peut dépasser 50 Mo, causant des lenteurs voire des erreurs de quota.	store/index.ts	CRITIQUE
2	Pas de nettoyage audio à la suppression — Quand un bouton est supprimé via <code>removeSoundButton</code> , l' <code>HTMLAudioElement</code> dans <code>audioRefs</code> n'est pas mis en pause/libéré, créant une fuite mémoire.	DetachedSoundboard.tsx	HAUTE
3	Rerenders excessifs via le state audio — L'événement <code>timeupdate</code> (toutes les ~250ms) appelle <code>setAudioStates</code> qui reconstruit un nouvel objet, entraînant un re-render de tout le composant. Avec 12 boutons jouant simultanément, cela devient coûteux.	DetachedSoundboard.tsx:81-84	HAUTE
4	Broadcast d'erreurs silencieux — <code>.catch(() => {})</code> ignore	DetachedSoundboard.tsx:52	MOYENNE

toutes les erreurs de broadcast,
rendant le debugging
impossible en production.

CSS avec valeurs dures

(hardcoded) — Les couleurs de fond (#ffffff, #f8f9fa) et les couleurs de texte (#495057) dans le CSS ignorent le thème sombre de l'application. En mode sombre, la fenêtre détachée reste blanche.

5

`DetachedSoundboard.css`

MOYENNE

Index comme identifiant

fragile — Les boutons sont identifiés par leur index dans la grille. Si l'utilisateur change cols/rows, les index des boutons ne correspondent plus à leur position visuelle.

6

`types/index.ts`

MOYENNE

Pas de retour d'erreur audio

— Si un fichier audio est corrompu ou impossible à charger, rien n'est affiché à l'utilisateur (pas de message d'erreur, pas d'état visuel).

7

`DetachedSoundboard.tsx`

MOYENNE

Absence de contrôle du volume dans la vue détachée

— Le volume est configurable uniquement dans le modal d'édition. Pas de slider rapide dans la fenêtre flottante.

8

`DetachedSoundboard.tsx`

BASSE

Latence télécommande pour les boucles

— Les sons en boucle ne sont pas synchronisés entre l'hôte et la télécommande. Un joueur peut entendre un son en boucle indéfini car il n'y a pas de mécanisme d'arrêt à distance.

9

`SoundboardRemote.tsx`

MOYENNE

Pas de limite de lecture

simultanée — Un clic rapide sur plusieurs boutons peut

- 10 lancer 12+ pistes audio simultanément, avec des risques de distorsion et de crash navigateur.

`DetachedSoundboard.tsx`**BASSE****Pas d'accessibilité (a11y) —**

Les boutons n'ont pas d' `aria-label`, la navigation au clavier (Tab, Enter, Escape) n'est pas optimisée, et l'état de lecture n'est pas annoncé aux lecteurs d'écran.

- 11 (Tab, Enter, Escape) n'est pas optimisée, et l'état de lecture n'est pas annoncé aux lecteurs d'écran.

`DetachedSoundboard.tsx`**BASSE****Propagation d'événements**

non maîtrisée —

`e.stopPropagation()` est

- 12 utilisé sur le bouton de fermeture mais pas sur les clics de bouton, ce qui peut interférer avec le drag.

`DetachedSoundboard.tsx:209`**BASSE**

3.2 Détail des problèmes critiques

● Problème #1 : Stockage base64 dans localStorage

Où : `src/store/index.ts` — Le middleware `persist` de Zustand sérialise tout l'état dans localStorage.

Impact :

- Quota localStorage limité à ~5-10 Mo par domaine
- Un fichier audio de 3 minutes en MP3 = ~2-3 Mo → ~3-4 Mo en base64
- Avec 12 boutons remplis, localStorage peut saturer
- La sérialisation/désérialisation du store devient lente

Solution recommandée : Stocker les fichiers audio séparément (blob IndexedDB), ne garder dans Zustand que les métadonnées + un ID de référence. Convertir base64 → Blob URL au chargement pour un playback performant.

● Problème #2 : Fuite mémoire sur suppression

Où : `src/store/index.ts:597-599` et `DetachedSoundboard.tsx`

Impact : `removeSoundButton(index)` retire le bouton du tableau, mais l' `HTMLAudioElement` correspondant dans `audioRefs.current` n'est ni mis en pause ni libéré. L'audio continue de jouer en mémoire.

Solution : Créer un `useEffect` qui surveille `soundboard.buttons` et nettoie les références audio orphelines.

4. Correctifs urgents (Quick Wins)

🔥 Correctif #1 : Nettoyage audio à la suppression de bouton

Effort : Faible

Ajouter un useEffect qui compare les index des boutons actuels avec les clés d'audioRefs, et nettoie les orphelins :

```
useEffect(() => {
  const activeIndices = new Set(soundboard.buttons.map(b => b.index));
  Object.keys(audioRefs.current).forEach(key => {
    const idx = parseInt(key);
    if (!activeIndices.has(idx)) {
      const audio = audioRefs.current[idx];
      audio.pause();
      audio.src = '';
      delete audioRefs.current[idx];
    }
  });
}, [soundboard.buttons]);
```

🔥 Correctif #2 : Optimisation des rerenders (timeupdate)

Effort : Moyen

Remplacer le state React pour les progress bars par des refs DOM directes, évitant les rerenders :

```
// Au lieu de setAudioStates, utiliser directement progressRefs
newAudio.addEventListener('timeupdate', () => {
  if (newAudio.duration) {
    const pct = (newAudio.currentTime / newAudio.duration) * 100;
    const bar = progressRefs.current.get(index);
    const btn = soundboard.buttons.find(b => b.index === index);
    if (bar) {
      bar.style.width = `${pct}%`;
      bar.style.backgroundColor = btn?.color || '#fff';
    }
  }
});
```

```
}  
});
```

🔥 Correctif #3 : Migration base64 → Blob URLs

Effort : Élevé

Solution en 3 phases :

1. **Conversion au chargement** : transformer toutes les chaînes base64 en Blob URLs au démarrage de l'app
2. **Stockage séparé** : utiliser IndexedDB via une lib comme [idb-keyval](#) pour stocker les fichiers audio bruts
3. **Migration** : script de conversion one-shot pour les utilisateurs existants

⚡ Correctif #4 : CSS Theme-aware

Effort : Faible

Remplacer les couleurs hardcodées par des variables CSS du thème :

```
.detached-soundboard {  
  background-color: hsl(var(--background));  
  border-color: hsl(var(--border));  
}  
.drag-handle {  
  background-color: hsl(var(--muted)) !important;  
  border-bottom-color: hsl(var(--border)) !important;  
}
```

5. Améliorations proposées

5.1 Améliorations fonctionnelles

A1 — Catégories et dossiers

Effort : Élevé**FONCTIONNEL**

Permettre d'organiser les sons en catégories (Ambiance, Combat, Exploration, Personnages...). Chaque catégorie serait un dossier dépliant dans la grille, avec un onglet dédié sur la télécommande.

A2 — Drag & drop pour réordonner les boutons

Effort : Moyen**FONCTIONNEL**

Permettre de réorganiser les boutons par glisser-déposer dans la grille. Actuellement, les boutons sont figés par index. Nouveau modèle : utiliser un ID unique (UUID) et un champ `position` ou un tableau ordonné.

A3 — Mixer audio embarqué

Effort : Moyen**UX**

Ajouter un panneau "Mixer" dans la fenêtre détachée avec :

- Slider de volume par piste active
- Bouton mute/solo par piste
- Indicateur VU-mètre (niveau audio en temps réel via AnalyserNode)

A4 — Mode Ambiance (lecture croisée)

Effort : Élevé**FONCTIONNEL**

Mode spécial qui permet de :

- Marquer des sons comme "ambiance" (lecture continue)
- Crossfade entre deux ambiances lors du changement
- Contrôle du volume global de l'ambiance
- État persisté : l'ambiance reprend là où elle s'était arrêtée

A5 — Enregistrement vocal direct

Effort : Moyen

FONCTIONNEL

Ajouter un bouton "Enregistrer" dans le modal d'édition qui utilise `MediaRecorder` + `getUserMedia` pour capturer le micro et l'affecter directement au bouton. Idéal pour les MJ qui veulent créer des voix de PNJ à la volée.

A6 — Raccourcis clavier

Effort : Faible

UX

Permettre d'assigner une touche clavier à chaque bouton (ex: 1-9, A-Z) pour un déclenchement rapide pendant la partie sans quitter le focus de la carte.

A7 — Import/Export de packs de sons

Effort : Moyen

FONCTIONNEL

Exporter la configuration complète de la soundboard (fichier JSON + sons) pour la partager ou la sauvegarder. Idéal pour des "soundpacks" thématiques partagés entre MJ.

5.2 Améliorations techniques

T1 — Web Audio API (AudioContext)

Effort : Élevé

TECHNIQUE

Remplacer HTMLAudioElement par l'API Web Audio (AudioContext, BufferSource) :

- Meilleur contrôle du gain (volume par piste + master)
- Analyse en temps réel (AnalyserNode pour VU-mètre)
- Effets audio (filtre passe-bas, réverb, égaliseur)
- Crossfading natif (GainNode avec ramp)
- Latence réduite

T2 — Limite de lecture simultanée

Effort : Faible

TECHNIQUE

Implémenter un pool de 4 voix maximum. Si une 5e piste est lancée, couper la plus ancienne ou la moins récente :

```
const MAX_VOICES = 4;
const playAudio = (index: number) => {
  const keys = Object.keys(audioRefs.current).map(Number);
  if (keys.length >= MAX_VOICES) {
    const oldest = keys[0]; // ou LRU
    audioRefs.current[oldest].pause();
    delete audioRefs.current[oldest];
  }
  // ... lancer la nouvelle piste
};
```

T3 — Synchronisation temps réel améliorée

Effort : Moyen

TECHNIQUE

Pour la télécommande :

- Diffuser l'état de lecture complet (index + currentTime) périodiquement
- Permettre l'arrêt à distance des boucles
- Afficher la progression sur la télécommande
- Indicateur "Son en cours chez le MJ" avec durée restante

T4 — Mode hors-ligne (Service Worker)

Effort : Élevé**TECHNIQUE**

Mettre en cache les fichiers audio via Service Worker pour que la soundboard fonctionne même sans connexion après le premier chargement. Critique pour les sessions en déplacement avec connexion instable.

T5 — Tests automatisés

Effort : Moyen**TECHNIQUE**

Ajouter des tests pour :

- Lecture/pause des sons
- Nettoyage des ressources audio
- Communication Realtime (mock Supabase)
- Comportement du drag & drop
- Déconnexion/reconnexion de la télécommande

T6 — Accessibilité (a11y)

Effort : Moyen**TECHNIQUE**

- `aria-label` sur chaque bouton avec le nom du son
- `aria-pressed` pour l'état de lecture
- `role="region"` et `aria-roledescription` pour la soundboard
- Navigation au clavier complète (Tab, Enter, Escape, flèches)
- Annonce des changements d'état via `aria-live`

6. Feuille de route priorisée

Phase 1 — Stabilisation (Semaine 1)

#	Tâche	Impact
1	Nettoyage audio orphelin à la suppression de bouton	● Critique / Faible effort
2	Migration base64 → Blob URLs + IndexedDB	● Critique / Effort élevé
3	Optimisation des rerenders (timeupdate via refs)	● Haute priorité
4	CSS theme-aware (suppression des couleurs hardcodées)	● Haute priorité

Phase 2 — Fonctionnalités clés (Semaine 2-3)

#	Tâche	Priorité
5	Raccourcis clavier	● Faible effort, fort impact
6	Limite de lecture simultanée (pool 4 voix)	● Protection anti-crash
7	Drag & drop pour réordonner les boutons	● Forte demande utilisateur
8	Mixer audio embarqué (volume par piste + master)	● Forte valeur ajoutée

Phase 3 — Avancé (Semaine 4-6)

#	Tâche	Priorité
9	Catégories et dossiers	● Organisation
10	Mode Ambiance (crossfade continu)	● Pour les puristes

11	Enregistrement vocal direct	● Créativité
12	Import/Export de packs	● Partage

Phase 4 — Excellence (Semaine 7-8)

#	Tâche	Priorité
13	Migration vers Web Audio API (AudioContext)	● Foundation technique
14	Synchronisation temps réel améliorée	● Qualité remote
15	Tests automatisés	● Qualité logicielle
16	Accessibilité (a11y)	● Inclusion
17	Mode hors-ligne (Service Worker)	● Robustesse

Estimation des charges

Estimation globale pour l'ensemble des améliorations :

- **Phase 1 (Stabilisation)** : ~3-4 jours
- **Phase 2 (Fonctionnalités clés)** : ~5-7 jours
- **Phase 3 (Avancé)** : ~8-10 jours
- **Phase 4 (Excellence)** : ~10-14 jours
- **Total** : ~26-35 jours ouvrés